

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO**

BRYAN RYAN
DAVI ALTOÉ
GABRIEL FARIA DIAS
WESLEY LEANDRO MOYA DA SILVA

TRABALHO DE BANCO DE DADOS

Trabalho apresentado à matéria de Banco de Dados da Faculdade Instituto Federal De Educação, Ciência e Tecnologia de São Paulo como requisito parcial para obtenção de nota para o 1º semestre de 2026.

Orientador: Doutor Murilo Vargas da Silva

SUMÁRIO

1. INTRODUÇÃO

- 1.1. OBJETIVO DO TRABALHO
- 1.2. ESCOLHA DO TEMA

2. REQUISITOS FUNCIONAIS

- 2.1. CADASTRO E AUTENTICAÇÃO DE JOGADORES
- 2.2. GERENCIAMENTO DE PERSONAGEM
- 2.3. GERENCIAMENTO DE INVENTÁRIO
- 2.4. GERENCIAMENTO DE ITENS
- 2.5. CRIAÇÃO E GERENCIAMENTO DE RAIDS
- 2.6. CONTROLE DE ACESSO
- 2.7. CONTROLE DE LOOT

3. FUNCTIONS, TRIGGERS E PROCEDURES

- 3.1. FUNCTIONS
- 3.2. TRIGGERS
- 3.3. PROCEDURES

4. DIAGRAMAS DO BANCO

- 4.1. MODELO CONCEITUAL
- 4.2. MODELO LÓGICO

1. INTRODUÇÃO

1.1. OBJETIVO DO TRABALHO

Foi proposto pelo professor Murilo Varges o trabalho de modelagem e desenvolvimento de um banco de dados Oracle utilizando linguagem PL/SQL no software SQL Developer para a matéria de banco de dados II.

A partir de um PDF disponibilizado pelo professor, tivemos acesso aos requisitos mínimos de implementação e a liberdade de escolha do tema do banco.

1.2. ESCOLHA DO TEMA

O tema escolhido pelo grupo foi a modelagem de um banco inspirado no jogo 'Escape From Tarkov'. Este se trata de um jogo online do gênero '*extraction shooter*' com grande enfoque em partidas contra outros jogadores e na exploração do mapa em busca de itens úteis e valiosos.

Dentre os vários motivos e fatores para a escolha do jogo, a de maior importância foi com relação aos itens e à participação dos jogadores nas partidas (também chamadas de Raids). Tendo em vista que, quando um jogador entra na raid com um conjunto de itens, esse conjunto está passível de sofrer alterações na quantidade, durabilidade ou simplesmente de sumir caso o jogador morra no meio da partida. Mas, caso ele consiga sair vivo do mapa, retornará com todos os itens que estão no seu inventário para sua base, especialmente os adquiridos durante a raid. Porém, se o tempo da raid acabar e o jogador não conseguir extrair (termo utilizado para sair do mapa), ele será declarado como 'perdido em ação', perdendo todos os itens do inventário. Novamente, isso demonstra a importância das transações durante o manejo dos itens dos jogadores tanto dentro, quanto fora das raids.

Diante deste cenário, vimos que a complexidade e a robustez requeridas na parte das transações do banco se encaixaram perfeitamente com os requisitos impostos pelo trabalho.

2. REQUISITOS FUNCIONAIS

Os requisitos funcionais descrevem as principais funcionalidades que esse sistema deve oferecer. No contexto desse banco de dados inspirado em jogos do gênero '*extraction shooter*', são descritas operações relacionadas ao gerenciamento

de jogadores (“players”), inventário, itens, partidas, transações e demais informações.

2.1. CADASTRO E AUTENTICAÇÃO DE USUÁRIOS

O banco deve armazenar dados da conta do jogador, como seu nome de usuário, data de criação da conta e o plano que o usuário adquiriu. Também devem ser registradas credenciais de acesso, como e-mail e senha.

2.2. GERENCIAMENTO DE PERSONAGEM

A conta criada pelo usuário tem um personagem associado, chamado “player” no contexto do banco, sendo este o que irá interagir com o jogo em si. São armazenadas informações relacionadas ao personagem, como sua experiência, facção e dinheiro, além de estatísticas como a quantidade de vezes que o player “matou”/“morreu” em uma partida, o chamado “kill/death ratio” ou simplesmente “k/d”.

Cada player também possui um inventário próprio, cujo tamanho e itens iniciais são baseados no plano que o usuário comprou, os itens que o player atualmente tem, possíveis banimentos impedindo o player de jogar.

2.3. GERENCIAMENTO DE INVENTÁRIO

O inventário trata-se da estrutura na qual os itens dos quais o player é proprietário são guardados. O inventário é usado não só para estabelecer uma relação de posse entre player e item, mas também define tamanho/peso do inventário, isto é, há um limite para a quantidade de itens que um usuário pode ter em qualquer dado momento. O tamanho máximo do inventário pode ser diferente dependendo do plano do usuário, sendo o plano premium (“PREMIUM”, no banco) mais espaçoso do que o inventário do plano padrão (“STANDARD”).

2.4. GERENCIAMENTO DE ITENS

O sistema permite o cadastro de novos itens contendo tamanho, peso, nome e valor.

2.5. CRIAÇÃO E GERENCIAMENTO DE RAIDS

O sistema deve permitir criar raids associadas a um mapa, informando horário de início, fim, limite de jogadores e o status atual (exemplo: caso a partida ainda

esteja acontecendo, o status será “RUNNING”). Também é feito o gerenciamento dessas raids, alterando seus status e atualizando as estatísticas de cada jogador ao fim da partida.

Além disso, a participação de um usuário em uma partida é feita através de uma tabela-relacionamento entre o player e a raid, criada apenas quando o jogador consegue entrar na partida. Tal estrutura permite armazenar dados específicos do jogador na partida.

2.6. CONTROLE DE ACESSO

Considerando a natureza de ser um jogo “online”, ou seja, todos os jogadores são pessoas reais, será implementado um sistema de banimento, onde jogadores poderão ser banidos temporariamente quando desrespeitarem as regras e atrapalhar a convivência de outros usuários. O banimento deve ser registrado com a data e horário onde o banimento ocorreu, a data de término e, por fim, o motivo que levou à punição. O banimento é usado para definir se um jogador está permitido entrar em partidas ou não.

2.7. CONTROLE DE LOOT

Ao final da partida, o sistema deve atualizar o inventário do jogador, adicionando os itens extraídos com sucesso em caso de vitória, e/ou removendo os itens perdidos durante a raid.

Tabela 1 - Requisitos funcionais

Requisito Funcional	Título	Descrição
RF01	Cadastro e autenticação de usuários	Permite cadastrar contas de usuários e armazenar suas credenciais, plano e informações pessoais para autenticação.
RF02	Gerenciamento de personagem	Gerencia os dados do personagem do jogador, incluindo progressão, estatísticas, moedas, inventário e informações relacionadas ao jogo.
RF03	Gerenciamento de inventário	Controla o inventário do jogador, seu tamanho, capacidade de armazenamento e os itens pertencentes

		ao jogador.
RF04	Gerenciamento de itens	Permite cadastrar e manter os atributos dos itens disponíveis no jogo, como nome, peso, tamanho e valor.
RF05	Criação e gerenciamento de raids	Gerencia a criação das partidas, participação dos jogadores, status da raid e atualização das estatísticas ao seu término.
RF06	Controle de acesso	Registra banimentos de usuários e impede que jogadores com punições ativas participem de novas partidas.
RF07	Controle de loot	Atualiza o inventário do jogador após a raid, adicionando os itens extraídos e removendo os itens perdidos durante a partida.

3. FUNCTIONS, TRIGGERS E PROCEDURES

3.1. FUNCTIONS

3.1.1. FN_MAPA_ALEATORIO

A função é usada para escolher um mapa aleatório dos mapas registrados no banco de dados. Usado principalmente no processo de criação de uma nova raid.

```
CREATE OR REPLACE FUNCTION fn_mapa_aleatorio
RETURN NUMBER
IS
    v_id_mapa MAPA.id_mapa%TYPE;
BEGIN
    SELECT id_mapa
    INTO v_id_mapa
    FROM (
        SELECT id_mapa
        FROM Mapa
        ORDER BY DBMS_RANDOM.VALUE
    )
    WHERE ROWNUM = 1;

    RETURN v_id_mapa;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
```

3.1.2. FN_CALCULAR_KD

Função utilizada para calcular o Kills/Deaths de um jogador e atualiza no registro do mesmo.

```
--function para registrar o K/D do jogador
CREATE OR REPLACE FUNCTION fn_calcular_kd(p_id_player NUMBER) RETURN NUMBER IS
    v_ratio NUMBER;
    v_kills NUMBER;
    v_deaths NUMBER;
    v_error EXCEPTION;
BEGIN
    SELECT kills, deaths INTO v_kills, v_deaths FROM PLAYER WHERE id_player = p_id_player;

    IF v_kills IS NULL AND v_deaths IS NULL THEN
        RAISE v_error;

    ELSE
        v_ratio := v_kills/v_deaths;
        UPDATE PLAYER SET kill_death_ratio = ROUND(v_ratio, 2) WHERE id_player = p_id_player;
        COMMIT;

        RETURN v_ratio;
    END IF;

EXCEPTION
    WHEN v_error THEN
        RAISE_APPLICATION_ERROR(-20001, 'VALORES NÃO ENCONTRADOS');

    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20002, 'ERRO AO CALCULAR A TAXA DE K/D');
END;
```

3.1.3. FN_CONVERTER_ITEM_PLAYER_RUBLO

Função que converte um item em dinheiro (rublo). Utilizada quando o jogador volta de uma raid com mais itens do que é capaz de guardar no estoque da base.

```
--function para transformar itens em dinheiro
CREATE OR REPLACE FUNCTION fn_converter_item_player_rublo(p_id_player NUMBER, p_id_item NUMBER, p_quantidade NUMBER) RETURN NUMBER IS
    v_total NUMBER;
    v_item_valor NUMBER;
BEGIN
    SELECT valor_item INTO v_item_valor FROM ITEM WHERE id_item = p_id_item;

    v_total := (p_quantidade*v_item_valor);

    UPDATE PLAYER SET rublo = (rublo+v_total) WHERE id_player = p_id_player;
    COMMIT;

    RETURN v_total;
END;
```

3.1.4. FN_ESPACO_LIVRE_ESTOQUE

Função responsável por calcular o espaço disponível no estoque da base do jogador.

```
--function para calcular o espaço livre do estoque
CREATE OR REPLACE FUNCTION fn_espaco_livre_estoque(p_id_inventario NUMBER) RETURN NUMBER IS
    v_espaco_estoque NUMBER;
    v_espaco_ocupado NUMBER;
BEGIN

    SELECT tamanho_inventario_base INTO v_espaco_estoque FROM INVENTARIO WHERE id_inventario = p_id_inventario;

    SELECT NVL(SUM(II.quantidade * I.tamanho_item), 0) INTO v_espaco_ocupado
        FROM INSTANCIA_ITEM II
        INNER JOIN ITEM I ON II.id_item = I.id_item
        WHERE II.id_inventario = p_id_inventario;

    RETURN (v_espaco_estoque - v_espaco_ocupado);
END;
```

3.2. TRIGGERS

3.2.1. TRG_VERIFICA_BANIMENTO

Trigger executada antes de ser feita uma inserção na tabela *Player_Raid*, isto é, realiza uma verificação se o jogador tem banimentos ativos ou não. Caso tenha, o jogador não é permitido entrada na partida, caso contrário, o INSERT ocorre normalmente.

```
CREATE OR REPLACE TRIGGER trg_verifica_banimento
BEFORE INSERT ON Player_Raid
FOR EACH ROW
DECLARE
    v_player_banido NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_player_banido
    FROM Banimento
    WHERE id_usuario = (
        SELECT p.id_usuario
        FROM Player p
        WHERE id_player = :NEW.id_player
    )
    AND (data_desbanimento IS NULL OR data_desbanimento > SYSDATE);

    IF v_player_banido > 0 THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Jogador está banido e não pode entrar em uma raid.'
        );
    END IF;
END;
```

3.2.2. TRG_VERIFICA_QUANT_PLAYERS_RAID

Trigger executada antes de um jogador entrar em uma raid. É verificada a quantidade de jogadores atualmente conectados na partida em que o usuário está tentando entrar. Caso haja vagas, o INSERT ocorre normalmente; caso contrário, um erro é ativado e o jogador não entra na partida.

```
CREATE OR REPLACE TRIGGER trg_verifica_quant_players_raid
BEFORE INSERT ON PLAYER_RAID
FOR EACH ROW
DECLARE
    v_quant_players NUMBER := 0;
    v_disponibilidade EXCEPTION;
BEGIN
    SELECT COUNT(*) INTO v_quant_players FROM PLAYER_RAID PLARAD WHERE PLARAD.id_raid = :NEW.id_raid;

    IF v_quant_players >= 16 THEN
        RAISE v_disponibilidade;
    END IF;
EXCEPTION
    WHEN v_disponibilidade THEN
        RAISE_APPLICATION_ERROR(-20001, 'Raid sem espaço para players');

    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20002, 'Erro ao inserir player');
END;
```

3.2.3. CHECAR_INVENTARIO_DISPONIVEL

Após o final de uma raid, quando os novos itens vão ser inseridos no estoque do jogador, a trigger checa se há espaço. Caso não haja, chama a função de converter para rublo, atualiza o montante na tabela do player e finaliza a trigger.

```

--Checa o espaço livre no inventário da base (Precisa dar uma mexida e implementar junto do espaço do inventário do jogador)
CREATE OR REPLACE TRIGGER checar_inventario_disponivel
BEFORE INSERT ON INSTANCIA_ITEM
FOR EACH ROW
DECLARE
    v_espaco_livre NUMBER;
    v_espaco_ocupado_item NUMBER;
    v_id_player NUMBER;
    v_valor_convertido NUMBER;
    v_error EXCEPTION;
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    v_espaco_livre := ESPACO_LIVRE_ESTOQUE(:NEW.id_inventario);

    SELECT tamanho_item INTO v_espaco_ocupado_item FROM ITEM WHERE id_item = :NEW.id_item;

    v_espaco_ocupado_item := v_espaco_ocupado_item * :NEW.quantidade;

    --Não há espaço para inserir o item no inventário
    IF v_espaco_ocupado_item > v_espaco_livre THEN
        RAISE v_error;
    END IF;
EXCEPTION
    WHEN v_error THEN
        SELECT id_player INTO v_id_player FROM INVENTARIO WHERE id_inventario = :NEW.id_inventario;
        v_valor_convertido := FN_CONVERTER_ITEM_PLAYER_RUBLO(v_id_player, :NEW.id_item, :NEW.quantidade);
        RAISE_APPLICATION_ERROR(
            -20001,
            'Capacidade máxima excedida. Transformando em rublo.'
        );
END;

```

3.3. PROCEDURES

3.3.1. SP_CRIAR_PLAYER

Procedure usada para criar um novo player. Quando o usuário compra um plano, seja o plano padrão ou premium, o seu “player” é adicionado ao banco. Também é criado o inventário que, como mencionado, tem seu tamanho definido através do plano assinado pelo usuário, além dos itens que o usuário recebe quando joga pela primeira vez. A procedure leva todos esses pontos em consideração quando adiciona um novo player no banco de dados, precisando receber com parâmetro apenas o id do usuário, o nome que o player escolheu para ser exibido e a facção que o jogador escolheu fazer parte.

```

CREATE OR REPLACE PROCEDURE sp_criar_player(p_id_usuario NUMBER, p_nome_player VARCHAR2, p_faccao VARCHAR2) IS
    v_cont NUMBER;
    v_id_player NUMBER;
    v_plano VARCHAR2(50);
    v_erro EXCEPTION;
BEGIN
    SELECT plano INTO v_plano FROM USUARIO WHERE id_usuario = p_id_usuario;
    --Checa se nao ha outro player nessa conta
    SELECT COUNT(*) INTO v_cont FROM PLAYER WHERE Player.id_usuario = p_id_usuario;
    --Verifica se nao ha outra conta e se o usuario possui o jogo pago
    IF v_cont = 0 AND v_plano != 'FREE' THEN

        --PADRAO
        IF v_plano = 'STANDARD' THEN
            INSERT INTO PLAYER(id_usuario, nome_player, faccao, rublo, dolar, euro)
            VALUES(p_id_usuario, p_nome_player, p_faccao, 50000, 2500, 3000);

            SELECT id_player INTO v_id_player FROM PLAYER WHERE id_usuario = p_id_usuario;

            INSERT INTO INVENTARIO(id_player, tamanho_inventario_base, tamanho_inventario_player, tamanho_slot_seguro, peso_max_player)
            VALUES (v_id_player, 300, 80, 4, 75);

        --PREMIUM
        ELSEIF v_plano = 'PREMIUM' THEN
            INSERT INTO PLAYER(id_usuario, nome_player, faccao, rublo, dolar, euro)
            VALUES(p_id_usuario, p_nome_player, p_faccao, 500000, 25000, 30000);

            SELECT id_player INTO v_id_player FROM PLAYER WHERE id_usuario = p_id_usuario;

            INSERT INTO INVENTARIO(id_player, tamanho_inventario_base, tamanho_inventario_player, tamanho_slot_seguro, peso_max_player)
            VALUES (v_id_player, 720, 80, 9, 75);
        END IF;

        COMMIT;

    ELSE
        RAISE v_erro;
    END IF;

EXCEPTION
    WHEN v_erro THEN
        RAISE_APPLICATION_ERROR(-20003, 'Usuario possui um plano inválido ou já possui conta vinculada.');
```

```

    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20004, 'Erro ao criar a conta do player.');
```

3.3.2. SP_TERMINAR_RAID

Recebe o id de uma raid como parâmetro e finaliza ela. Além disso, também atualiza os status de todos os jogadores que estão ativos na partida quando ela se encerra, visto que é logicamente impossível um jogador permanecer na partida quando acaba. Jogadores que não conseguiram escapar a tempo antes do fim da partida são tidos como “MIA”, sigla para “Missing in Action” ou “Perdido em Ação”.

```

CREATE OR REPLACE PROCEDURE sp_terminar_raid (
    p_id_raid Raid.id_raid%TYPE
) IS
    CURSOR c_player IS
        SELECT
            pr.id_player_raid,
            pr.id_raid
        FROM Player_Raid pr
        JOIN Raid r ON pr.id_raid = r.id_raid
        WHERE pr.status_player = 'IN ACTION';

    v_player_rec c_player%ROWTYPE;

    v_raid_encontrada NUMBER;
    v_raid_existe EXCEPTION;
BEGIN
    SELECT COUNT(*)
    INTO v_raid_encontrada
    FROM Raid WHERE id_raid = p_id_raid;

    IF v_raid_encontrada = 0 THEN
        RAISE v_raid_existe;
    END IF;

    OPEN c_player;
    FETCH c_player INTO v_player_rec;
    WHILE (c_player%FOUND) LOOP
        UPDATE Player_Raid
        SET status_player = 'MIA'
        WHERE status_player = 'IN ACTION'
            AND id_player_raid = v_player_rec.id_player_raid
            AND id_raid = v_player_rec.id_raid;
    END LOOP;

    UPDATE Raid
    SET status_raid = 'FINISHED'
    WHERE status_raid = 'RUNNING'
        AND id_raid = p_id_raid;

    COMMIT;

```

3.3.3. SP_PROCESSA_STATUS_PLAYER

Um jogador durante uma partida pode ter diferentes status, como vivo, morto, ou perdido em ação. Quando ocorre essa mudança de status, vários atributos e estatísticas do banco devem ser mudadas e atualizadas, trabalho realizado por essa procedure. Quando um jogador é morto ou perdido em ação, os itens que ele atualmente tem equipado são removidos do seu inventário além do seu k/d global ser atualizado, o jogador responsável por matar o jogador também recebe alteração nas suas estatísticas, sendo o número de mortes causadas aumentada em 1.

```

CREATE OR REPLACE PROCEDURE sp_processa_status_player(
    p_id_player Player.id_player%TYPE,
    p_id_raid    Raid.id_raid%TYPE,
    p_status    Player_Raid.status_player%TYPE
)
IS
    v_scav Player_Raid.scav%TYPE;
BEGIN
    UPDATE Player_Raid
    SET status_player = p_status
    WHERE id_player = p_id_player
        AND id_raid = p_id_raid;

    SELECT scav
    INTO v_scav
    FROM Player_Raid
    WHERE id_player = p_id_player
        AND id_raid = p_id_raid;

    UPDATE Player p
    SET kills = kills + (
        SELECT pr.kills
        FROM Player_Raid pr
        WHERE pr.id_player = p_id_player
            AND pr.id_raid = p_id_raid
    )
    WHERE id_player = p_id_player;

    IF p_status IN ('KIA', 'MIA') THEN
        UPDATE Player
        SET deaths = deaths + 1
        WHERE id_player = p_id_player;

        IF v_scav = 0 THEN
            DELETE FROM Instancia_Item
            WHERE id_inventario = (
                SELECT id_inventario
                FROM Inventario
                WHERE id_player = p_id_player
            )
            AND equipado = 1;

```

3.3.4. SP_CRIAR_RAID

Uma procedure responsável por criar uma raid a partir do id do mapa e o horário no qual a raid vai estar ocorrendo (horário no mundo do jogo).

```

--procedure para criar uma raid
CREATE OR REPLACE PROCEDURE sp_criar_raid(p_id_mapa NUMBER, p_horario_raid VARCHAR2) IS
BEGIN
    INSERT INTO RAID(id_mapa, inicio_raid, final_raid, horario_raid)
    VALUES(p_id_mapa, TO_CHAR(SYSDATE, 'DD/MM/YYYY - HH24:MI:SS'), TO_CHAR(SYSDATE+30/1440, 'DD/MM/YYYY - HH24:MI:SS'), p_horario_raid);

    COMMIT;
END;

```

3.3.5. SP_ENTRAR_PLAYER_RAID

Procedure responsável por inserir um player em uma raid em andamento.

```

--procedure para inserir o jogador na raid
CREATE OR REPLACE PROCEDURE sp_entrar_player_raid(p_id_player NUMBER, p_id_raid NUMBER, p_scav NUMBER) IS
BEGIN
    INSERT INTO Player_Raid(id_player, id_raid, scav)
    VALUES(p_id_player, p_id_raid, p_scav);

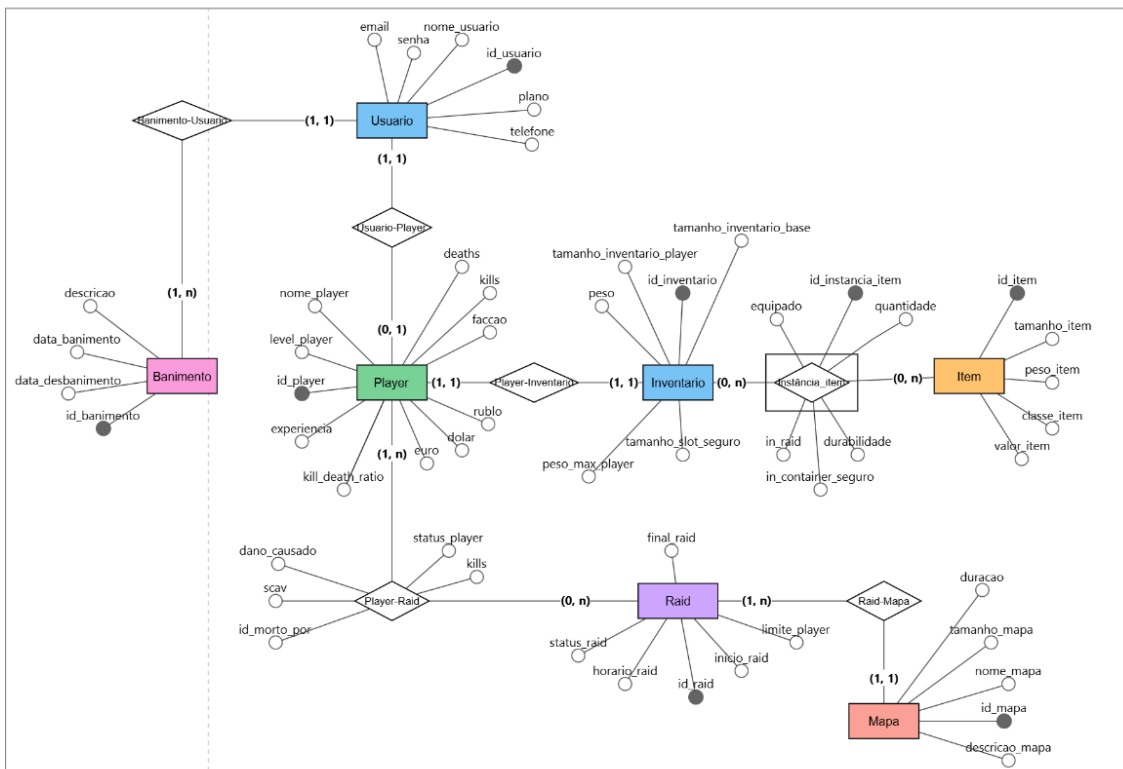
    COMMIT;
END;

```

4. DIAGRAMAS DO BANCO

Todos os diagramas foram feitos utilizando as ferramentas online do site BRModelo.

4.1. Modelo Conceitual



4.2. Modelo Lógico

